21. Memento Design Pattern in Java

21.1 Introduction

The **Memento Design Pattern** is a **behavioral pattern** used to capture and restore an object's internal state without violating encapsulation.

21.2 Problem it Solves

It enables undo/rollback capabilities by storing snapshots of the object's internal state.

21.3 What is Memento Pattern?

It involves three components:

- **Originator**: The object whose state needs to be saved and restored
- **Memento**: Stores the state
- **Caretaker**: Manages saved states without altering them

21.4 Real-World Analogy

A text editor's undo feature, where previous states are saved and can be restored.

21.5 Structure

```
package com.mannemlokesh;

import java.util.ArrayList;
import java.util.List;

public class Memento {
	private String state;

	public Memento(String state) {
		this.state = state;
	}

	public String getState() {
		return state;
	}
}

class Originator {
	private String state;

	public void setState(String state) {
		this.state = state;
	}

	public String getState() {
		return state;
	}

	public Memento saveStateToMemento() {
		return new Memento(state);
	}

	public void getStateFromMemento(Memento memento) {
```

```
            state = memento.getState();
        }
}

class Caretaker {
        private List<Memento> mementoList = new ArrayList<>();

        public void add(Memento state) {
                mementoList.add(state);
        }

        public Memento get(int index) {
                return mementoList.get(index);
        }
}
```

21.6 Benefits

- Provides undo/rollback support
- Preserves encapsulation
- Easy to implement

21.7 Implementation Steps

1. Create a Memento to store internal state
2. Modify Originator to use memento for save/restore
3. Use Caretaker to store and retrieve mementos

21.8 Examples

Example1: Text Editor Undo Feature

```
package com.mannemlokesh.example1;

import java.util.ArrayList;
import java.util.List;

class TextEditor {
        private String content;

        public void setContent(String content) {
                this.content = content;
        }

        public String getContent() {
                return content;
        }

        public EditorMemento save() {
                return new EditorMemento(content);
        }

        public void restore(EditorMemento memento) {
                this.content = memento.getContent();
        }
}
```

```
class EditorMemento {
	private final String content;

	public EditorMemento(String content) {
		this.content = content;
	}

	public String getContent() {
		return content;
	}
}

class UndoManager {
	private final List<EditorMemento> history = new ArrayList<>();

	public void backup(EditorMemento memento) {
		history.add(memento);
	}

	public EditorMemento undo() {
		if (history.size() > 1) {
			history.remove(history.size() - 1);
		}
		return history.get(history.size() - 1);
	}
}

public class Example1TextEditorClient {
	public static void main(String[] args) {
		TextEditor editor = new TextEditor();
		UndoManager undoManager = new UndoManager();

		editor.setContent("First Version");
		undoManager.backup(editor.save());

		editor.setContent("Second Version");
		undoManager.backup(editor.save());

		editor.setContent("Third Version");
		undoManager.backup(editor.save());

		System.out.println("Current: " + editor.getContent());
		editor.restore(undoManager.undo());
		System.out.println("After Undo: " + editor.getContent());
	}
}
```

Output:

```
Current: Third Version
After Undo: Second Version
```

Example2: Game Save/Restore State

```
package com.mannemlokesh.example2;

class Game {
	private int level;
```

```
	private int health;

	public void set(int level, int health) {
		this.level = level;
		this.health = health;
	}

	public void printState() {
		System.out.println("Level: " + level + ", Health: " + health);
	}

	public GameMemento save() {
		return new GameMemento(level, health);
	}

	public void load(GameMemento memento) {
		this.level = memento.getLevel();
		this.health = memento.getHealth();
	}
}

class GameMemento {
	private final int level;
	private final int health;

	public GameMemento(int level, int health) {
		this.level = level;
		this.health = health;
	}

	public int getLevel() {
		return level;
	}

	public int getHealth() {
		return health;
	}
}

public class Example2GameClient {
	public static void main(String[] args) {
		Game game = new Game();
		game.set(1, 100);
		GameMemento save1 = game.save();

		game.set(2, 75);
		game.printState();

		game.load(save1);
		game.printState();
	}
}
```

Output:

```
Level: 2, Health: 75
Level: 1, Health: 100
```

Example3: Multiple Undo/Redo Support

```
package com.mannemlokesh.example3;

import java.util.Stack;

class Document {
	private String text;

	public void setText(String text) {
		this.text = text;
	}

	public String getText() {
		return text;
	}

	public DocumentMemento save() {
		return new DocumentMemento(text);
	}

	public void restore(DocumentMemento memento) {
		text = memento.getText();
	}
}

class DocumentMemento {
	private final String text;

	public DocumentMemento(String text) {
		this.text = text;
	}

	public String getText() {
		return text;
	}
}

class HistoryManager {
	private Stack<DocumentMemento> undoStack = new Stack<>();
	private Stack<DocumentMemento> redoStack = new Stack<>();

	public void save(DocumentMemento memento) {
		undoStack.push(memento);
		redoStack.clear();
	}

	public DocumentMemento undo(Document current) {
		if (!undoStack.isEmpty()) {
			redoStack.push(current.save());
			return undoStack.pop();
		}
		return current.save();
	}

	public DocumentMemento redo(Document current) {
		if (!redoStack.isEmpty()) {
			undoStack.push(current.save());
			return redoStack.pop();
		}
		return current.save();
```

```
	}
}

public class Example3DocumentClient {
	public static void main(String[] args) {
		Document doc = new Document();
		HistoryManager history = new HistoryManager();

		doc.setText("Version 1");
		history.save(doc.save());

		doc.setText("Version 2");
		history.save(doc.save());

		doc.setText("Version 3");
		System.out.println("Current: " + doc.getText());

		doc.restore(history.undo(doc));
		System.out.println("Undo: " + doc.getText());

		doc.restore(history.redo(doc));
		System.out.println("Redo: " + doc.getText());
	}
}
```

Output:

```
1 2 3 4 5
```

21.9 Memento vs Other Patterns

Pattern	Purpose
Memento	Capture and restore object state
Command	Encapsulate operation as an object
Observer	Notify objects on state changes

21.10 Pros and Cons

✅ **Pros**

- Preserves encapsulation
- Enables undo functionality
- Simple and intuitive

❌ **Cons**

- Can use more memory for storing states
- Caretaker must manage memory or state list

21.11 When to Use / Not to Use

Use When:

- You want undo/redo functionality
- You need to restore state without exposing details

Avoid When:

- System state is large and expensive to copy

21.12 Summary

Feature	Description
Intent	Save and restore internal object state
Use Case	Undo/Redo, Game save, Editor state
Key Benefit	Encapsulation of state with reversibility

The **Memento Pattern** is ideal for scenarios requiring undo, rollback, or recovery without violating encapsulation.